

置信度调度与机密性门控组件 — 更新草稿

2026-09-20 · @Someone

0. 范围与来源

本草稿更新的是架构 v0.3 中「confidence scoring」与「gateway & policy」两个组件的交互设计，合并三方面输入：

1. **第2次客户会议（9/15）的决定**——置信度信号选型：token log-probability + Semantic Entropy Probe (SEP)，排除模型自评置信度；置信度决定是否需要升级，敏感度类别决定是否允许升级。
2. **本次内部讨论确定的门控机制**——机密性检查是无条件先行的硬门控 (hard gate)，置信度调度只在门控允许的空间内起作用，二者不做加权合并。
3. **Yudi Guo 《Security and De-identification》调研报告**——对当前「二元敏感度门」设计提出了直接、有实证支持的修正意见，以及三条需要并入设计的额外约束（请求前定策略、累计会话预算、确定性授权层）。

下文逐节说明每部分内容，以及三者合并后哪些地方需要调整现有设计。

1. 置信度信号（已决定，无变化）

第2次客户会议已锁定信号选型，本次讨论未改动这一部分，仅重申作为后续架构的输入：

- **两个信号，8B本地模型上近零额外成本**：模型本身产生的token log-probability，以及在模型隐藏状态上一次性训练好的Semantic Entropy Probe (SEP)。两者在7-8B规模上都有已发表的实证支持。
- **排除模型自评置信度**：跨7个3-9B开源模型测试，91.7%的回答无论对错都声称至少95%的置信度，因此不采用「直接问模型有多确信」这一路线。
- **主模型 Llama 3.1 8B，副模型 Qwen3 8B**（代码与多语言查询），用Hugging Face transformers以4-bit运行，便于读取log-probs和隐藏状态。
- **职责划分**：置信度分数只回答「要不要升级」，不回答「能不能升级」——能不能升级完全由第2节的机密性门控决定，两者不做加权合并。

2. 机密性门控机制（本次讨论确定）

核心原则：机密性等级是硬约束 (gate)，置信度是硬约束允许范围内的连续调度 (route)；两者不合并成一个分数，也不能互相替代。

- **门控无条件先行、无条件执行**：每条请求先查一次敏感度标签，成本是 $O(1)$ 的categorical lookup，几乎不产生额外开销；不能因为置信度高就跳过这一步——置信度信号可能被分布外样本或对抗性输入误导，一旦门控依赖置信度触发，隐私保护就可能被绕过。
- **门控结果分三档**：
 - `ALLOW_CLOUD` ——可直接调用云端
 - `SANITIZE_THEN_CLOUD` ——脱敏（PII/实体替换）后才能调用云端
 - `LOCAL_ONLY` ——禁止出境，无论置信度多低都不升级
- **LOCAL_ONLY 场景下置信度低的兜底手段**（成本递增）：本地多次采样做self-consistency → 本地更大模型 → 本地RAG检索 → 显式标注不确定性直接作答 → 人工复核队列。
- **审计与合规**：门控判定必须对每条请求都留痕，不能因为置信度高就没有合规判定记录；机密性分级同时还应控制日志/telemetry里是否记录完整prompt等非升级类行为，不只是控制是否升级。

这一节的判断此前是基于通用工程原则给出的，第3节会说明它与Yudi的调研有哪些吻合、哪些需要修正。

3. 合并后的两级决策架构

第1节（置信度）与第2节（门控）合并后的请求处理顺序：

1. **第一级：机密性门控**——查数据敏感度标签，无条件执行，与请求内容质量无关。输出三档策略之一。
2. **第二级：置信度调度**——只在门控允许的空间内运行：
 - 门控= `ALLOW_CLOUD`：本地模型先答，SEP+softmax组合分数低于阈值则升级云端（阈值按敏感等级×任务类型分别校准，建议用conformal risk control而非固定阈值）。
 - 门控= `SANITIZE_THEN_CLOUD`：置信度低于阈值时，先脱敏再发送，只把脱敏后的部分发给云端。
 - 门控= `LOCAL_ONLY`：置信度分数不再决定「是否升级」，只决定「本地用哪种兜底手段」（见第2节）。

这与9/15会议架构图v0.3的表述一致（置信度决定是否需要升级，敏感度类别决定是否允许升级；两者都不满足时，系统承认不确定而不是发送机密数据），本节是对这一表述的操作化展开。

门控先行、置信度后行的请求流程如下：

```

flowchart TD
    Req["用户请求"] --> Gate["机密性门控（无条件先行）"]
    Gate -->|ALLOW_CLOUD| AC{"置信度达标?"}
    Gate -->|SANITIZE_THEN_CLOUD| SC{"置信度达标?"}
    Gate -->|LOCAL_ONLY| LC{"置信度达标?"}
    AC -->|是| Reply["本地直接回复"]
    AC -->|否| Cloud["升级云端"]
    SC -->|是| Reply
    SC -->|否| San["脱敏后升级云端"]
    LC -->|是| Reply
    LC -->|否| Fallback["本地兜底：集成/更大模型/RAG/人工复核"]

    classDef gate fill:#241c12,stroke:#d4a24c,stroke-width:2px,color:#f0d9a8
    classDef decision fill:#1b2333,stroke:#6b7690,stroke-width:1.5px,color:#e8ecf1
    classDef local fill:#122018,stroke:#3f9463,stroke-width:1.5px,color:#a9e4c2
    classDef cloud fill:#17233c,stroke:#5c86c7,stroke-width:1.5px,color:#a9c4ef
    classDef fallback fill:#2a1d10,stroke:#d98a3d,stroke-width:1.5px,color:#f0c894
    classDef entry fill:#1b2333,stroke:#48536b,stroke-width:1.5px,color:#e8ecf1

    class Req entry
    class Gate gate
    class AC,SC,LC decision
    class Reply local
    class Cloud,San cloud
    class Fallback fallback

```

三条分支共享同一套置信度打分逻辑，区别只在于「置信度不达标时能去哪」——这正是第2节「gate先行、route在后」的图形化版本。

4. 关键修正：不能只用「敏感度」驱动三档门控

Yudi的调研直接针对第2节的三档门控提出了修正，需要采纳：

- **MINIM (ICML 2026) 的消融实验**：把门控从二元/单一敏感度信号换成两个分数——固有**敏感度**与任务条件下的**必要性**——驱动Keep/Abstract/Remove三种动作。实验显示：仅用敏感度驱动门控，任务关键内容保留率会崩到**0.0401**（助手基本不可用）；仅用必要性驱动，敏感信息泄露率是两分数方案的约2倍；两个分数联合驱动时任务保留0.9491、泄露0.1010，明显更均衡。

- **对我们设计的含义：第2节的三档策略**

(`ALLOW_CLOUD` / `SANITIZE_THEN_CLOUD` / `LOCAL_ONLY`) 目前只由数据的固有敏感度标签决定，没有引入「这条信息对当前任务有多必要」这一维度——这正是MINIM实验中「仅敏感度」这一档，也是实验里表现最差的一档。需要在门控里补上必要性评分，让三档判断变成敏感度×必要性的联合结果，而不是敏感度单独决定。

- **重要限制（不可直接套用）：**MINIM的评分器是在Web UI可访问性树上训练的图神经网络，针对Gmail/Reddit/购物网站等场景，训练用的评分器本身对我们的文本场景没有直接可迁移性。能迁移的是**策略形状**（两分数、三个及以上动作）和其消融实验证明的方向，不是具体模型或阈值（论文给出的 $\tau_{\text{necessity}}=1.0$ 、 $\tau_{\text{sensitivity}}=5.0$ 是在其原始0-10量表下的数值，不能直接套用）。
- **必要性评分从哪来：**调研4.2节指出，必要性应该从用户问题本身推导，敏感度应该来自数据来源（provenance）——两者都不需要读取检索到的内容，因此评分层本身不在prompt注入的攻击面内。这一点应作为我们必要性打分模块的设计前提：打分必须在检索发生前完成，检索结果只能填槽，不能扩大已经确定的出境权限。

5. 需要并入设计的另外三条约束

除第4节的核心修正外，调研报告还提出三条与本组件直接相关、目前设计里缺失的约束：

1. **判定必须先于检索（plan before retrieve）**——目前的设计示例中，本地模型先读取检索到的10-Q文件，再决定哪部分可以出境；这意味着一个在文件里植入指令的攻击者可以影响「什么可以出境」这个判断（confused-deputy模式）。修正后的不变量：意图、必要性/敏感度评分、出境许可，全部只从用户原始问题推导，且在检索开始前完成；检索结果只能填充已定义的槽位，不能扩大许可范围。这与第4节「评分层不在注入攻击面内」是同一件事的两个表述。
2. **授权必须是确定性代码，不能是模型**——模型可以提议放行，但只有确定性代码能批准放行。已有实证支持：四智能体流水线在无防御时，伪造审批注入的攻击100%得逞；加入独立授权层后，验证智能体被绕过的比例仍是100%，但由此导致的不安全动作发生率降到0%（因为授权不依赖被绕过的那个组件）。同类实验在智能体浏览器场景下，攻击成功率从68-100%降到所有测试模型上的0%。这条建议我们的门控实现必须是allowlist形式的确定性代码，而不是让LLM自行判断「这次能不能发」。

3. **要限制单次会话的累计出境量，不只是单条请求**——连续20个问题各自都不含持仓、身份或策略信息，每条都能通过门控，但整个序列拼起来仍会暴露完整持仓结构。这是逐请求过滤无法捕捉的问题。报告提出两种思路：按受保护话题设置会话额度、逐次扣减（简单但两头都不准，即高估重复提问、低估真正收窄范围的那次披露）；或维护一组「诱饵」备选方案，每次放行后让对手模型猜测哪个是真实答案，按其相对猜中概率的提升计费（更贴近真实风险，但两种方法都不提供可证明的上界，只是需要明确写出的限制，不能当作已解决的保证）。这条目前完全不在我们的架构里，属于净增项，建议列入开放风险而非本期实现范围。

6. 开放风险与待验证事项

- **必要性评分模块尚未设计**——第4节要求的「必要性」打分目前只是原则，具体怎么从用户问题推导出必要性分数、用什么模型或规则，还没有方案，需要单独立项。
- **累计会话预算未解决**——第5.3节两种思路都不提供可证明的上界；按会话刷新额度会被「开新会话」绕过，永不刷新又会让助手逐渐不可用，报告原文也承认「目前没有好答案」。
- **不可逆变换的边界**——任何必须出现在最终回复里的字段，脱敏时用的变换必须是可逆的，这条约束要在设计阶段就锁定，否则回复阶段会出不了正确答案。
- **金融场景的适用性未经验证**——调研中审阅的方法均未在金融数据上验证过，我们的场景中「秘密」是交易方向和敞口，而不是身份本身，报告作者明确把这一点标注为推断性、非结论性的迁移。
- **门控与置信度的组合尚未联调测试**——本文档给出的是设计层面的合并，第4-5节的修正尚未反映在原型代码里，需要在下一次迭代中落地并纳入5.1节的四项独立测试（检测质量、端到端泄露、任务效用、注入抵抗）。

参考来源

- 第2次客户会议纪要（9/15），Architecture v0.3
- Yu, H. et al. MINIM: Privacy-Aware Minimal View for Agents via Trusted Local Sanitization. ICML 2026. arXiv:2606.13949
- Safin, T. H. et al. Trust Propagation and Structural Containment in Multi-agent LLM Pipelines. IEEE CARS 2026. arXiv:2609.17648
- Irshad, H. et al. The Verifiable Action Card. arXiv:2609.18411
- Xie, J., Li, S. OCELOT: Inference-Leakage Budgets for Privacy-Preserving LLM Agents. arXiv:2606.12341
- Yudi Guo, Security and De-identification（CMU MISM Capstone 团队工作包，2026-09-19）——完整方法论、指标定义与测试计划见原文档